

PROPUESTA DE POBLADO DE DATOS MULTI-TENANT PARA CRTLPyme

Fecha: 12 de Noviembre, 2025

Proyecto: CRTLPyme - Plataforma SaaS para gestión de PyMEs

Objetivo: Corregir y estructurar correctamente el poblado de datos respetando la arquitectura multi-tenant

ÍNDICE

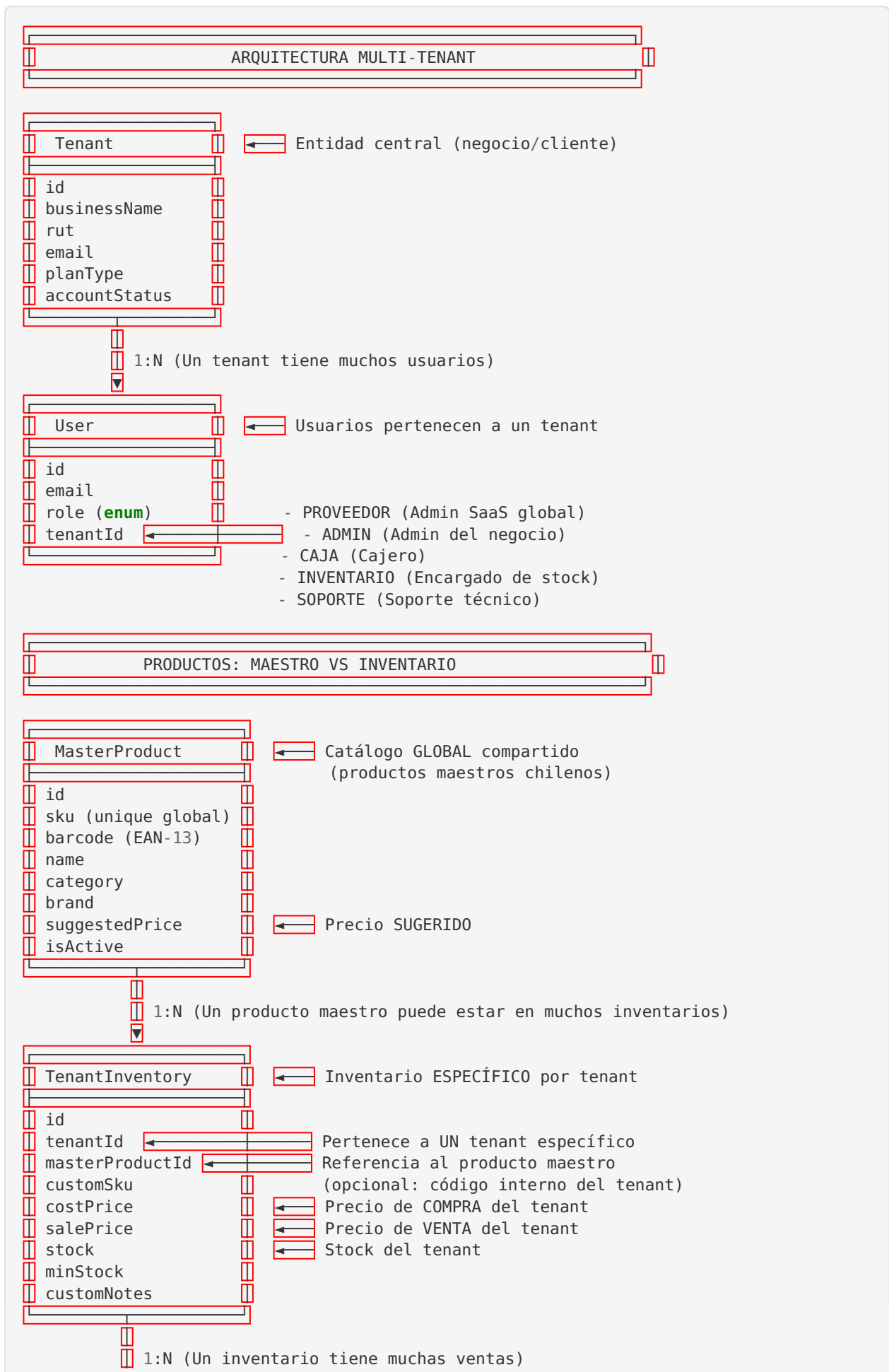
1. [Análisis de la Estructura Actual](#)
 2. [Problemas Identificados](#)
 3. [Arquitectura Multi-Tenant Correcta](#)
 4. [Propuesta de Implementación](#)
 5. [Estructura de Datos Propuesta](#)
 6. [Plan de Ejecución Paso a Paso](#)
 7. [Código de Ejemplo](#)
 8. [Verificaciones Post-Seed](#)
-

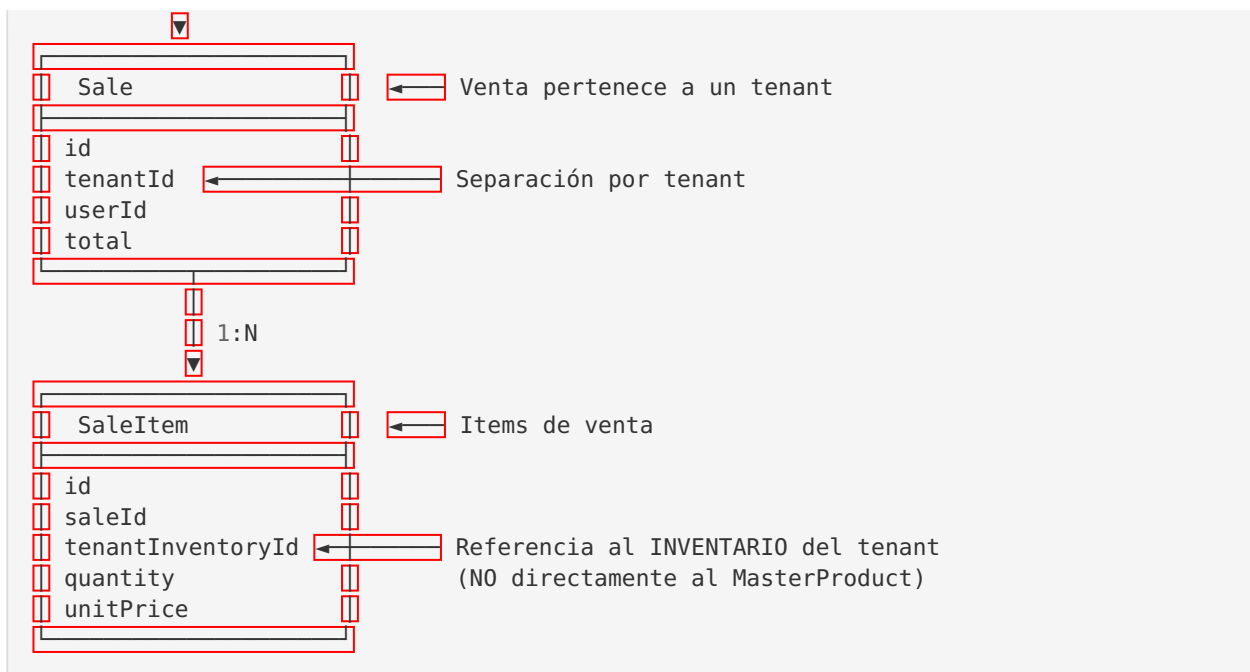
1. ANÁLISIS DE LA ESTRUCTURA ACTUAL

1.1 Schema de Prisma - Arquitectura Multi-Tenant

El schema de Prisma está **BIEN DISEÑADO** y sigue correctamente el patrón multi-tenant:

Entidades Principales:





Separación Multi-Tenant:

✓ Correcta:

- Cada `User` tiene `tenantId` (todos los usuarios pertenecen a un tenant)
- Cada `TenantInventory` tiene `tenantId` (inventario aislado por tenant)
- Cada `Sale` tiene `tenantId` (ventas aisladas por tenant)
- `SaleItem` referencia a `TenantInventory` (no a `MasterProduct` directamente)

✓ Rol PROVEEDOR:

- Es el administrador de la plataforma SaaS
- Tiene acceso a TODOS los tenants
- Puede ver datos agregados de todos los clientes
- Pertenecer a un tenant especial "plataforma"

Tabla Legacy:

⚠ **DEPRECATED:** La tabla `Product` está marcada como `products_legacy` y **NO DEBE USARSE**. Es para mantener compatibilidad con código antiguo.

1.2 Archivos de Seed Existentes

Se encontraron **5 archivos de seed** en el proyecto:

Archivo	Ubicación	Propósito	Estado
seed.ts	/prisma/seed.ts	Seed principal (ejecutado por <code>npm run seed</code>)	❌ PROBLEMÁTICO
seed-complete.ts	/prisma/seed-complete.ts	Seed completo con 500 productos maestros	⚠️ CASI CORRECTO
seed-multitenancy.ts	/prisma/seed-multitenancy.ts	Seed multi-tenant con 4 negocios	❌ PROBLEMÁTICO
seed-subscription-plans.ts	/prisma/seed-subscription-plans.ts	Seed de planes de suscripción	❌ CON BUG
seed-comprehensive.ts	/prisma/seed-comprehensive.ts	(No revisado en detalle)	⚠️ DESCONOCIDO

2. PROBLEMAS IDENTIFICADOS

● PROBLEMA 1: seed.ts usa tabla LEGACY

Archivo: /prisma/seed.ts (ejecutado por defecto con `npm run seed`)

Problema:

```
// Línea 74 - USA LA TABLA LEGACY ❌
await prisma.product.create({
  data: {
    sku: sku,
    barcode: producto.ean13,
    name: producto.name,
    // ...
    tenantId: tenant.id, // ❌ Productos atados directamente al tenant
  },
})
```

Impacto:

- ❌ No usa `MasterProduct` (catálogo global)
- ❌ No usa `TenantInventory` (inventario por tenant)
- ❌ Los productos se crean directamente para un solo tenant
- ❌ No hay separación de catálogo maestro vs inventario
- ❌ Otros tenants no pueden acceder a estos productos

Adicionalmente:

- Busca el archivo `data/productos_chilenos.json` que **NO EXISTE**

● PROBLEMA 2: seed-multitenancy.ts usa tabla LEGACY

Archivo: /prisma/seed-multitenancy.ts

Problema:

```
// Línea 134 - USA LA TABLA LEGACY ❌
await prisma.product.create({
  data: {
    ...product,
    costPrice: product.costPrice.toString(),
    salePrice: product.salePrice.toString(),
    minStock: 5,
    isActive: true,
    tenantId: tenant1.id, // ❌ Productos atados directamente al tenant
  },
})
```

Impacto:

- ❌ Crea productos únicos para cada tenant (no hay catálogo compartido)
- ❌ Minimarket Los Andes tiene productos de minimarket
- ❌ Ferretería El Tornillo tiene productos de ferretería
- ❌ Pero no hay un catálogo maestro común
- ❌ No pueden buscar productos del maestro para agregar a su inventario

● PROBLEMA 3: seed-subscription-plans.ts usa YEARLY en vez de ANNUAL

Archivo: /prisma/seed-subscription-plans.ts

Problema:

```
// Líneas 124, 149, 177, 207 - USA 'YEARLY' ❌
{
  name: 'Plan Básico - Anual',
  billingCycle: 'YEARLY' as BillingCycle, // ❌ INCORRECTO
  // ...
}
```

Definición en Schema:

```
// schema.prisma - línea 761
enum BillingCycle {
  MONTHLY
  QUARTERLY
  ANNUAL // ✅ CORRECTO (no YEARLY)
}
```

Impacto:

- ❌ Los planes anuales no se crean correctamente
- ❌ Causa el error “No hay planes disponibles en este momento” en la landing page
- ❌ El frontend espera ANNUAL pero recibe YEARLY

Nota: Este problema ya fue corregido en `PricingPlans.tsx` anteriormente, pero el seed sigue creando planes con `YEARLY`.

● PROBLEMA 4: seed-complete.ts busca archivo inexistente

Archivo: `/prisma/seed-complete.ts`

Problema:

```
// Línea 304 - Busca archivo que NO EXISTE ✗
const productosPath = path.join('/home/ubuntu', 'productos_chile.json')

if (!fs.existsSync(productosPath)) {
  console.error(`✗ No se encontró el archivo ${productosPath}`)
  return []
}
```

Verificación:

```
$ ls -lh /home/ubuntu/productos_chile.json
Archivo no encontrado
```

Impacto:

- ✗ El seed no puede importar productos maestros
 - ✗ La función `importarProductos()` retorna un array vacío
 - ✗ No se crean productos en `MasterProduct`
 - ✗ Por lo tanto, no se pueden crear inventarios en `TenantInventory`
 - ✗ Todo el seed falla o queda incompleto
-

● PROBLEMA 5: Configuración del seed principal

Archivo: `package.json`

Configuración actual:

```
"scripts": {
  "seed": "tsx prisma/seed.ts", // ✗ Ejecuta el seed problemático
  "seed:complete": "tsx prisma/seed-complete.ts",
  "seed:multitenancy": "tsx prisma/seed-multitenancy.ts",
  // ...
}
```

Impacto:

- ✗ El comando por defecto `npm run seed` ejecuta el seed problemático
 - ✗ Los desarrolladores no saben cuál seed usar
 - ✗ Hay confusión sobre cuál es el seed correcto
-

3. ARQUITECTURA MULTI-TENANT CORRECTA

3.1 Conceptos Clave

A. Productos Maestros (Master Products)

Son el **catálogo centralizado** de productos chilenos que facilita el proceso de registro.

Características:

- ☒ Globales (no pertenecen a ningún tenant específico)
- ☒ Compartidos por TODOS los tenants
- ☒ Contienen información base: nombre, marca, categoría, EAN-13
- ☒ Tienen precio **SUGERIDO** (referencial)
- ☒ Son de **solo lectura** para clientes
- ☒ Gestionados por el rol PROVEEDOR

Flujo de uso:

1. Cliente busca producto en el catálogo maestro
2. Cliente agrega producto a su inventario
3. Sistema sugiere precio de compra/venta
4. Cliente modifica precios según su negocio

B. Inventarios por Tenant (TenantInventory)

Cada cliente tiene su **PROPIO inventario** aislado.

Características:

- ☒ Pertenece a UN tenant específico
- ☒ Referencia a un producto del catálogo maestro
- ☒ Tiene precios personalizados (costo y venta)
- ☒ Tiene stock propio
- ☒ Puede tener SKU interno personalizado
- ☒ Totalmente aislado de otros tenants

Reglas de aislamiento:

```
-- ☒ CORRECTO: Filtrar por tenantId
SELECT * FROM tenant_inventory WHERE tenantId = 'tenant123'

-- ☒ INCORRECTO: Ver inventario de otros tenants
SELECT * FROM tenant_inventory -- Sin filtro
```

C. Rol PROVEEDOR (Admin SaaS)

El proveedor es **CRTLPyne como empresa**, no un cliente.

Permisos:

- ☒ Ver datos de TODOS los clientes
- ☒ Ver total de ventas por cliente
- ☒ Filtrar y seleccionar cualquier cliente
- ☒ Ver stock de cualquier cliente
- ☒ Gestionar catálogo maestro de productos
- ☒ Crear/editar/eliminar planes de suscripción
- ☒ Suspender/activar cuentas
- ☒ Ver métricas globales de la plataforma

Restricciones:

-  NO puede modificar inventario de clientes
 -  NO puede crear ventas para clientes
 -  Solo puede ver/analizar datos
-

3.2 Flujo de Datos Multi-Tenant

FLUJO DE DATOS

1. CREACIÓN DEL CATÁLOGO MAESTRO (una sola vez)

PROVEEDOR crea productos maestros

- Coca-Cola 1.5L
- Pan Hallulla
- Leche Colun 1L
- Arroz Tucapel 1kg
- ... (500+ productos chilenos)



Tabla: master_products (global)

- sku: único global
- barcode: EAN-13
- suggestedPrice: referencial

2. CADA TENANT AGREGA PRODUCTOS A SU INVENTARIO

TENANT 1: Minimarket Don Luis

Admin busca en catálogo maestro

Selecciona: Coca-Cola 1.5L

Sistema sugiere precio: \$1200

Admin modifica: \$1000 costo, \$1500 venta |

Ingresar stock: 50 unidades



Tabla: tenant_inventory

- tenantId: "tenant1"
- masterProductId: "coca-cola-15"
- costPrice: 1000
- salePrice: 1500
- stock: 50

TENANT 2: Almacén El Rinconcito

Admin busca en catálogo maestro

Selecciona: Coca-Cola 1.5L

(MISMO PRODUCTO MAESTRO)

Admin modifica: \$900 costo, \$1400 venta |

Ingresar stock: 30 unidades



Tabla: tenant_inventory

- tenantId: "tenant2"
- masterProductId: "coca-cola-15"
- costPrice: 900
- salePrice: 1400
- stock: 30

3. VENTAS POR TENANT (aisladas)

TENANT 1 realiza una venta

```
Sale
• tenantId: "tenant1"
• userId: user del tenant1
• total: $4500
  └─ SaleItem
    • tenantInventoryId: inv-tenant1
    • quantity: 3
```

- ✗ TENANT 2 NO PUEDE ver esta venta
- ✓ PROVEEDOR SÍ PUEDE ver esta venta (acceso global)

4. VISTA DEL PROVEEDOR (global)

```
Dashboard Admin SaaS

Total Tenants: 13
Total Ventas Hoy: $1,234,567

Ventas por Cliente:
• Minimarket Don Luis: $45,000
• Almacén El Rinconcito: $32,000
• Supermercado Familiar: $78,000

[Filtrar por cliente] ▼
→ Minimarket Don Luis
  Stock: 50 productos
  Ventas mes: $450,000
```

4. PROPUESTA DE IMPLEMENTACIÓN

4.1 Solución Recomendada

Opción A: Usar y Mejorar seed-complete.ts ★ RECOMENDADO

seed-complete.ts es el más cercano a la arquitectura correcta, pero necesita:

1. ✓ Crear archivo productos_chile.json con productos chilenos
2. ✓ Corregir seed-subscription-plans.ts (YEARLY → ANNUAL)
3. ✓ Ejecutar npm run seed:subscription-plans primero
4. ✓ Ejecutar npm run seed:complete --clean después
5. ✓ Cambiar el script por defecto en package.json

Opción B: Crear nuevo seed desde cero

Si prefieres empezar limpio, crear un nuevo seed-production.ts con:

1. Planes de suscripción (con ANNUAL)
2. Productos maestros desde un JSON/CSV
3. Usuario PROVEEDOR
4. 3-4 tenants de ejemplo
5. Usuarios por tenant
6. Inventarios por tenant (referenciando productos maestros)

- 7. Ventas históricas
 - 8. Suscripciones activas
-

4.2 Requerimientos de Datos

A. Archivo de Productos Maestros

Necesitas un archivo con productos chilenos reales. Formato JSON:

```
{
  "productos": [
    {
      "ean13": "7800123456789",
      "nombre": "Coca-Cola Original 1.5L",
      "marca": "Coca-Cola",
      "categoria": "Bebidas",
      "precio_compra": 800,
      "precio_venta": 1200
    },
    {
      "ean13": "7800234567890",
      "nombre": "Pan Hallulla Ideal",
      "marca": "Ideal",
      "categoria": "Panadería",
      "precio_compra": 300,
      "precio_venta": 500
    }
    // ... más productos
  ]
}
```

¿De dónde sacar los productos?

- API pública de productos chilenos (si existe)
 - Scraping de sitios como Jumbo, Lider, Walmart Chile
 - Base de datos de códigos de barras chilenos
 - Dataset existente de productos de retail chileno
 - Crear manualmente 100-200 productos básicos
-

5. ESTRUCTURA DE DATOS PROPUESTA

5.1 Ejemplo de Datos

```
// =====
// 1. PLANES DE SUSCRIPCIÓN (8 planes)
// =====
SubscriptionPlan:
- Plan Gratuito (MONTHLY, $0)
- Plan Básico - Mensual (MONTHLY, $19,990)
- Plan Profesional - Mensual (MONTHLY, $39,990)
- Plan Empresarial - Mensual (MONTHLY, $79,990)
- Plan Básico - Anual (ANNUAL, $191,904) ☐ Corregir a ANNUAL
- Plan Profesional - Anual (ANNUAL, $383,904) ☐ Corregir a ANNUAL
- Plan Empresarial - Anual (ANNUAL, $767,904) ☐ Corregir a ANNUAL
- Plan Premium - Anual (ANNUAL, $1,199,904) ☐ Corregir a ANNUAL

// =====
// 2. CATÁLOGO MAESTRO (500 productos)
// =====
MasterProduct:
- id: "mp001"
  sku: "7800123456789"
  barcode: "7800123456789"
  name: "Coca-Cola Original 1.5L"
  category: "Bebidas"
  brand: "Coca-Cola"
  suggestedPrice: 1200

- id: "mp002"
  sku: "7800234567890"
  barcode: "7800234567890"
  name: "Pan Hallulla Ideal"
  category: "Panadería"
  brand: "Ideal"
  suggestedPrice: 500

// ... 498 productos más

// =====
// 3. TENANT ESPECIAL: PLATAFORMA
// =====
Tenant:
- id: "platform-tenant"
  businessName: "CRTLPyME - Plataforma"
  rut: "99.999.999-9"
  email: "plataforma@crtlpyme.com"
  planType: ENTERPRISE
  accountStatus: ACTIVE

User (PROVEEDOR):
- id: "admin-saas"
  email: "admin@crtlpyme.com"
  password: "Admin2025!" (hasheado)
  firstName: "Admin"
  lastName: "Plataforma"
  role: PROVEEDOR ☐ Acceso global
  tenantId: "platform-tenant"

// =====
// 4. TENANTS (13 negocios de ejemplo)
// =====

// TENANT 1: Minimarket Don Luis
Tenant:
- id: "t001"
```

```

businessName: "Minimarket Don Luis"
rut: "76.123.456-7"
email: "contacto@minimarketdonluis.cl"
planType: PRO
accountStatus: ACTIVE

```

User (Admin):

```

- email: "admin@minimarketdonluis.cl"
  password: "Demo2025!"
  role: ADMIN
  tenantId: "t001"

```

User (Cajero):

```

- email: "caja@minimarketdonluis.cl"
  password: "Demo2025!"
  role: CAJA
  tenantId: "t001"

```

User (Inventario):

```

- email: "inventario@minimarketdonluis.cl"
  password: "Demo2025!"
  role: INVENTARIO
  tenantId: "t001"

```

TenantInventory (Inventario del negocio):

```

- tenantId: "t001"
  masterProductId: "mp001" (Coca-Cola 1.5L)
  costPrice: 900
  salePrice: 1300
  stock: 50
  minStock: 10

- tenantId: "t001"
  masterProductId: "mp002" (Pan Hallulla)
  costPrice: 350
  salePrice: 550
  stock: 100
  minStock: 20

```

// ... 50-200 productos más en su inventario

Subscription:

```

- tenantId: "t001"
  planId: "Plan Pro - Mensual"
  status: ACTIVE
  startDate: 6 meses atrás
  nextBillingDate: próximo mes
  lifetimeValue: $119,940 (6 x $19,990)

```

Sale (Ventas históricas):

```

- id: "sale001"
  tenantId: "t001"
  userId: "caja@minimarketdonluis.cl"
  saleNumber: "000001"
  total: $3900
  paymentMethod: CASH
  items:
    - tenantInventoryId: "inv-t001-mp001"
      quantity: 3
      unitPrice: 1300

```

// ... 50-300 ventas en los últimos 3 meses

```
// TENANT 2: Almacén El Rinconcito
Tenant:
- id: "t002"
  businessName: "Almacén El Rinconcito"
  rut: "76.234.567-8"
  email: "elrinconcito@gmail.com"
  planType: BASIC
  accountStatus: ACTIVE

User (Admin):
- email: "admin@elrinconcito.com"
  role: ADMIN
  tenantId: "t002"

User (Cajero):
- email: "caja@elrinconcito.com"
  role: CAJA
  tenantId: "t002"

TenantInventory:
- tenantId: "t002"
  masterProductId: "mp001" (MISMO producto: Coca-Cola)
  costPrice: 850 ☐ Diferente precio que Tenant 1
  salePrice: 1250 ☐ Diferente precio que Tenant 1
  stock: 30 ☐ Diferente stock

// ... productos propios del inventario

// ... TENANTS 3-13 (similar estructura)
```

6. PLAN DE EJECUCIÓN PASO A PASO

6.1 Preparación

Paso 1: Crear archivo de productos maestros

Opción A: Crear manualmente (100 productos básicos)

```
cd /home/ubuntu
mkdir -p data
nano productos_chile.json
```

Contenido inicial (ejemplo con 10 productos):

```

{
  "productos": [
    {
      "ean13": "7800123456789",
      "nombre": "Coca-Cola Original 1.5L",
      "marca": "Coca-Cola",
      "categoria": "Bebidas",
      "precio_compra": 800,
      "precio_venta": 1200
    },
    {
      "ean13": "7800234567890",
      "nombre": "Pan Hallulla Ideal",
      "marca": "Ideal",
      "categoria": "Panadería",
      "precio_compra": 300,
      "precio_venta": 500
    },
    {
      "ean13": "7800345678901",
      "nombre": "Leche Entera Colun 1L",
      "marca": "Colun",
      "categoria": "Lácteos",
      "precio_compra": 700,
      "precio_venta": 1000
    },
    {
      "ean13": "7800456789012",
      "nombre": "Arroz Grado 1 Tucapel 1kg",
      "marca": "Tucapel",
      "categoria": "Abarrotes",
      "precio_compra": 600,
      "precio_venta": 900
    },
    {
      "ean13": "7800567890123",
      "nombre": "Aceite Vegetal Chef 900ml",
      "marca": "Chef",
      "categoria": "Abarrotes",
      "precio_compra": 1200,
      "precio_venta": 1800
    },
    {
      "ean13": "7800678901234",
      "nombre": "Huevos Rojos x12 Santa Isabel",
      "marca": "Santa Isabel",
      "categoria": "Lácteos",
      "precio_compra": 1500,
      "precio_venta": 2200
    },
    {
      "ean13": "7800789012345",
      "nombre": "Detergente En Polvo Omo 1kg",
      "marca": "Omo",
      "categoria": "Limpieza",
      "precio_compra": 2000,
      "precio_venta": 2800
    },
    {
      "ean13": "7800890123456",
      "nombre": "Papel Higiénico Elite x4",
      "marca": "Elite",

```

```

    "categoria": "Higiene",
    "precio_compra": 1800,
    "precio_venta": 2500
  },
  {
    "ean13": "7800901234567",
    "nombre": "Fideos Carozzi Cabello Ángel 400g",
    "marca": "Carozzi",
    "categoria": "Abarrotes",
    "precio_compra": 500,
    "precio_venta": 800
  },
  {
    "ean13": "7801012345678",
    "nombre": "Azúcar Granulada Iansagro 1kg",
    "marca": "Iansagro",
    "categoria": "Abarrotes",
    "precio_compra": 700,
    "precio_venta": 1100
  }
]
}

```

Nota: Necesitas expandir esto a 200-500 productos. Puedes:

- Buscar datasets de productos chilenos en Kaggle/GitHub
- Usar ChatGPT para generar más productos con EAN-13 ficticios
- Contactar con proveedores de datos de retail en Chile

Opción B: Usar un archivo existente o generar programáticamente

Si encuentras una fuente de datos, ajustar el formato al esperado.

Paso 2: Corregir seed-subscription-plans.ts

```

cd /home/ubuntu/CRTLPyME
nano prisma/seed-subscription-plans.ts

```

Cambios a realizar:

```

// BUSCAR (4 ocurrencias: líneas 124, 149, 177, 207):
billingCycle: 'YEARLY' as BillingCycle,

// REEMPLAZAR POR:
billingCycle: 'ANNUAL' as BillingCycle,

```

Comando rápido (si tienes acceso a sed):

```

cd /home/ubuntu/CRTLPyME
sed -i "s/billingCycle: 'YEARLY'/billingCycle: 'ANNUAL'/g" prisma/seed-subscription-plans.ts

```

Paso 3: Modificar seed-complete.ts para usar el archivo correcto

```
nano prisma/seed-complete.ts
```

Cambio en línea 304:

```
// ANTES:
const productosPath = path.join('/home/ubuntu', 'productos_chile.json')

// DESPUÉS (si creaste el archivo en /home/ubuntu/data/):
const productosPath = path.join('/home/ubuntu', 'data', 'productos_chile.json')

// O MEJOR (relativo al proyecto):
const productosPath = path.join(process.cwd(), 'data', 'productos_chile.json')
```

También ajustar el parseo del JSON (línea 311-312):

```
// ANTES:
const data = JSON.parse(fs.readFileSync(productosPath, 'utf-8'))
const productosSeleccionados = data.slice(0, limite)

// DESPUÉS:
const data = JSON.parse(fs.readFileSync(productosPath, 'utf-8'))
const productosSeleccionados = data.productos.slice(0, limite) // ←
Agregar .productos
```

Paso 4: Cambiar el seed por defecto en package.json

```
nano package.json
```

Cambio:

```
// ANTES:
"scripts": {
  "seed": "tsx prisma/seed.ts",
  // ...
}

// DESPUÉS:
"scripts": {
  "seed": "tsx prisma/seed-complete.ts", // ← Cambiar al seed correcto
  "seed:legacy": "tsx prisma/seed.ts",   // ← Renombrar el anterior
  // ...
}
```

6.2 Ejecución del Seed

Paso 5: Respaldar la base de datos (IMPORTANTE)

```
# Conectar a Cloud SQL
gcloud sql connect crtlypyme-db --user=postgres --project=crtlypyme-477300

# Dentro de PostgreSQL:
\l                # Listar bases de datos
\c crtlypyme_db    # Conectar a la base de datos

# Hacer backup de tablas críticas (opcional)
\copy tenants TO '/tmp/tenants_backup.csv' CSV HEADER;
\copy users TO '/tmp/users_backup.csv' CSV HEADER;
```

Paso 6: Ejecutar el seed de planes de suscripción

```
cd /home/ubuntu/CRTLPyme
npm run seed:subscription-plans
```

Salida esperada:

```
🚀 Iniciando seed de planes de suscripción...
✅ Plan creado: Plan Gratuito - $0 /mes
✅ Plan creado: Plan Básico - Mensual - $19,990 /mes
✅ Plan creado: Plan Profesional - Mensual - $39,990 /mes
✅ Plan creado: Plan Empresarial - Mensual - $79,990 /mes
✅ Plan creado: Plan Básico - Anual - $191,904 /año
✅ Plan creado: Plan Profesional - Anual - $383,904 /año
✅ Plan creado: Plan Empresarial - Anual - $767,904 /año
✅ Plan creado: Plan Premium - Anual - $1,199,904 /año

✅ Seed completado exitosamente!
📊 Resumen:
  - Planes creados: 8
  - Total: 8
```

Paso 7: Ejecutar el seed completo

CON LIMPIEZA (recomendado en desarrollo):

```
cd /home/ubuntu/CRTLPyme
npm run seed:complete -- --clean
```

El flag `--clean` eliminará:

- Ventas
- Items de venta
- Ajustes de stock
- Inventarios por tenant
- Productos maestros
- Sesiones de caja

- Pagos de suscripciones
- Suscripciones
- Usuarios
- Tenants

⚠ NO eliminará:

- Planes de suscripción (se crean si no existen)

SIN LIMPIEZA (recomendado en producción):

```
npm run seed:complete
```

Esto agregará datos sin eliminar los existentes (usando `upsert` donde sea posible).

Salida esperada:

 Iniciando población completa de datos para CRTLPyme

=====

 Limpiando base de datos...

 Base de datos limpiada

 Creando planes de suscripción...

 Planes ya existen, omitiendo creación...

 3 planes de suscripción creados

 Importando 500 productos desde archivo JSON...

 500 productos importados

 Creando usuario administrador de plataforma...

 Usuario administrador **creado**:

Email: **admin@crtlpyme.com**

Contraseña: Admin2025

RoL: PROVEEDOR (Super **Admin**)

 Creando negocios PyME chilenos...

 13 negocios creados

 Creando usuarios para cada negocio...

 35 usuarios creados

Contraseña para **todos**: Demo2025

 Creando inventario para cada negocio...

 1,450 productos en inventario creados

 Creando suscripciones para cada negocio...

 13 suscripciones creadas

 Creando ventas históricas (últimos 3 meses)...

 1,847 ventas históricas creadas

 Creando movimientos de inventario...

 143 movimientos de inventario creados

=====

 POBLACIÓN DE DATOS COMPLETADA EXITOSAMENTE 

=====

 RESUMEN:

-  Planes de **suscripción**: 3
-  Productos en catálogo **maestro**: 500
-  Negocios **PyME**: 13
-  Usuarios **totales**: 36 (incluye **admin**)
-  Productos en **inventario**: 1,450
-  Suscripciones **activas**: 13
-  Ventas **históricas**: 1,847
-  Movimientos de **inventario**: 143

=====

 CREDENCIALES DE **ACCESO**:

 Administrador de **Plataforma**:

Email: **admin@crtlpyme.com**

Contraseña: Admin2025❗
 Rol: PROVEEDOR (Super Admin)

👤 Usuarios de Negocios:
 Contraseña para todos: Demo2025❗
 Email: admin@[dominio-del-negocio]

💡 PRÓXIMOS PASOS:

1. Ejecutar migraciones: `npm run prisma migrate dev`
2. Iniciar la aplicación: `npm run dev`
3. Acceder a `http://localhost:3000`
4. Iniciar sesión con las credenciales del administrador

7. CÓDIGO DE EJEMPLO

7.1 Snippet de Creación de Productos Maestros

```
// Crear productos en el catálogo maestro (MasterProduct)
async function crearProductosMaestros(productos: any[]) {
  const productosCreados = []

  for (const prod of productos) {
    const masterProduct = await prisma.masterProduct.upsert({
      where: {
        sku: prod.ean13 || `SKU-${Date.now()}-${Math.random()}`
      },
      update: {
        name: prod.nombre,
        barcode: prod.ean13,
        category: prod.categoria,
        brand: prod.marca,
        suggestedPrice: prod.precio_venta,
        isActive: true
      },
      create: {
        sku: prod.ean13 || `SKU-${Date.now()}-${Math.random()}`,
        barcode: prod.ean13,
        name: prod.nombre,
        category: prod.categoria,
        brand: prod.marca,
        suggestedPrice: prod.precio_venta,
        unit: 'unidad',
        isActive: true
      }
    })

    productosCreados.push(masterProduct)
  }

  return productosCreados
}
```

7.2 Snippet de Creación de Inventario por Tenant

```
// Crear inventario para un tenant (TenantInventory)
async function crearInventarioTenant(
  tenant: Tenant,
  productosMaestros: MasterProduct[]
) {
  const inventarios = []

  // Cada tenant selecciona 50-200 productos del catálogo maestro
  const numProductos = Math.floor(Math.random() * 150) + 50
  const productosSeleccionados = productosMaestros
    .sort(() => Math.random() - 0.5)
    .slice(0, numProductos)

  for (const masterProduct of productosSeleccionados) {
    // Cada tenant define sus propios precios (basados en el sugerido)
    const costoBase = Number(masterProduct.suggestedPrice) * 0.6
    const variacion = (Math.random() * 0.2 - 0.1) // -10% a +10%
    const costPrice = Math.round(costoBase * (1 + variacion))

    const margen = 0.3 + Math.random() * 0.3 // 30% a 60%
    const salePrice = Math.round(costPrice * (1 + margen))

    const stock = Math.floor(Math.random() * 95) + 5

    // Verificar si ya existe en el inventario
    const existente = await prisma.tenantInventory.findFirst({
      where: {
        tenantId: tenant.id,
        masterProductId: masterProduct.id
      }
    })

    if (!existente) {
      const inventario = await prisma.tenantInventory.create({
        data: {
          tenantId: tenant.id,
          masterProductId: masterProduct.id,
          costPrice,
          salePrice,
          stock,
          minStock: 5 + Math.floor(Math.random() * 10),
          isActive: true
        }
      })

      inventarios.push(inventario)
    }
  }

  return inventarios
}
```

7.3 Snippet de Verificación de Separación Multi-Tenant

```
// Verificar que los inventarios estén correctamente aislados
async function verificarAislamientoMultiTenant() {
  const tenants = await prisma.tenant.findMany()

  for (const tenant of tenants) {
    // Obtener inventario del tenant
    const inventario = await prisma.tenantInventory.findMany({
      where: { tenantId: tenant.id },
      include: { masterProduct: true }
    })

    console.log(`\nTenant: ${tenant.businessName}`)
    console.log(`  Productos en inventario: ${inventario.length}`)

    // Verificar que NO haya productos de otros tenants
    const productosOtrosTenants = await prisma.tenantInventory.findMany({
      where: {
        masterProductId: { in: inventario.map(i => i.masterProductId) },
        tenantId: { not: tenant.id }
      }
    })

    console.log(`  Mismo producto en otros tenants: ${productosOtrosTenants.length}`)
    console.log(`  ✅ Separación correcta: ${productosOtrosTenants.length > 0 ?`
'Sí' : 'No'}`)
  }
}
```

7.4 Query de Ejemplo: Vista del PROVEEDOR

```
// Vista global del PROVEEDOR: ver ventas de todos los tenants
async function obtenerVentasGlobales(session: Session) {
  // Verificar que el usuario sea PROVEEDOR
  if (session.user.role !== 'PROVEEDOR') {
    throw new Error('Acceso denegado')
  }

  // PROVEEDOR puede ver TODOS los tenants
  const tenants = await prisma.tenant.findMany({
    include: {
      sales: {
        include: {
          items: {
            include: {
              tenantInventory: {
                include: {
                  masterProduct: true
                }
              }
            }
          }
        }
      }
    }
  })

  // Calcular ventas por tenant
  const ventasPorTenant = tenants.map(tenant => ({
    tenantId: tenant.id,
    businessName: tenant.businessName,
    totalVentas: tenant.sales.reduce((sum, sale) => sum + Number(sale.total), 0),
    numeroVentas: tenant.sales.length
  }))

  return ventasPorTenant
}
```

7.5 Query de Ejemplo: Vista del ADMIN de Tenant

```
// Vista de ADMIN: ver solo su propio tenant
async function obtenerVentasPropias(session: Session) {
  // ADMIN solo puede ver su propio tenant
  if (session.user.role !== 'ADMIN' && session.user.role !== 'PROVEEDOR') {
    throw new Error('Acceso denegado')
  }

  const tenantId = session.user.tenantId

  // Filtrar por tenantId
  const ventas = await prisma.sale.findMany({
    where: { tenantId }, // ← CRÍTICO: filtrar por tenant
    include: {
      items: {
        include: {
          tenantInventory: {
            include: {
              masterProduct: true
            }
          }
        }
      }
    }
  })

  return ventas
}
```

8. VERIFICACIONES POST-SEED

8.1 Verificaciones en Base de Datos

```
# Conectar a la base de datos
gcloud sql connect ctrlpyme-db --user=postgres --project=ctrlpyme-477300
\c ctrlpyme_db
```

Verificación 1: Planes de Suscripción

```
-- Debe haber 8 planes (4 MONTHLY + 4 ANNUAL)
SELECT
  name,
  billing_cycle,
  price,
  is_active
FROM subscription_plans
ORDER BY sort_order;

-- Resultado esperado:
-- Plan Gratuito          | MONTHLY | 0          | t
-- Plan Básico - Mensual  | MONTHLY | 19990      | t
-- Plan Profesional - Mensual | MONTHLY | 39990      | t
-- Plan Empresarial - Mensual | MONTHLY | 79990      | t
-- Plan Básico - Anual    | ANNUAL  | 191904     | t
-- Plan Profesional - Anual | ANNUAL  | 383904     | t
-- Plan Empresarial - Anual | ANNUAL  | 767904     | t
-- Plan Premium - Anual   | ANNUAL  | 1199904    | t
```

⚠ Si ves **YEARLY** en lugar de **ANNUAL**, el seed NO está corregido.

Verificación 2: Productos Maestros

```
-- Debe haber productos en el catálogo maestro
SELECT COUNT(*) as total_productos_maestros
FROM master_products;

-- Resultado esperado: 500 (o el número que configuraste)

-- Ver algunos productos
SELECT
  sku,
  barcode,
  name,
  category,
  brand,
  suggested_price
FROM master_products
LIMIT 10;
```

⚠ Si la tabla está vacía, el archivo **productos_chile.json** no se encontró o tiene formato incorrecto.

Verificación 3: Tenants

```
-- Debe haber al menos 13 tenants + 1 de plataforma
SELECT
    business_name,
    rut,
    email,
    plan_type,
    account_status
FROM tenants
ORDER BY business_name;

-- Resultado esperado:
-- CRTLPyme - Plataforma      | 99.999.999-9 | plataforma@crtlpyme.com | ENTERPRISE |
ACTIVE
-- Minimarket Don Luis      | 76.123.456-7 | contacto@...           | PRO        |
ACTIVE
-- Almacén El Rinconcito    | 76.234.567-8 | elrinconcito@gmail.com  | BASIC      |
ACTIVE
-- ... (11 más)
```

Verificación 4: Usuarios y Roles

```
-- Debe haber 1 PROVEEDOR + usuarios por tenant (2-3 por tenant)
SELECT
    role,
    COUNT(*) as cantidad
FROM users
GROUP BY role;

-- Resultado esperado:
-- PROVEEDOR | 1
-- ADMIN     | 13
-- CAJA      | 10-13
-- INVENTARIO | 10-13

-- Verificar el usuario PROVEEDOR
SELECT
    email,
    first_name,
    last_name,
    role
FROM users
WHERE role = 'PROVEEDOR';

-- Resultado esperado:
-- admin@crtlpyme.com | Admin | Plataforma | PROVEEDOR
```

Verificación 5: Inventarios por Tenant

```
-- Cada tenant debe tener productos en su inventario
SELECT
  t.business_name,
  COUNT(ti.id) as productos_inventario
FROM tenants t
LEFT JOIN tenant_inventory ti ON t.id = ti.tenant_id
WHERE t.rut != '99.999.999-9' -- Excluir tenant de plataforma
GROUP BY t.id, t.business_name
ORDER BY t.business_name;

-- Resultado esperado:
-- Minimarket Don Luis      | 87
-- Almacén El Rinconcito    | 124
-- Supermercado Familiar    | 156
-- ... (cada uno con 50-200 productos)
```

⚠ Si algún tenant tiene 0 productos, el inventario no se creó correctamente.

Verificación 6: Separación Multi-Tenant

```
-- Verificar que un producto maestro está en múltiples inventarios
-- (esto es CORRECTO y demuestra la arquitectura)

SELECT
  mp.name AS producto_maestro,
  COUNT(DISTINCT ti.tenant_id) AS num_tenants,
  string_agg(DISTINCT t.business_name, ', ') AS tenants_que_lo_tienen
FROM master_products mp
JOIN tenant_inventory ti ON mp.id = ti.master_product_id
JOIN tenants t ON ti.tenant_id = t.id
WHERE t.rut != '99.999.999-9'
GROUP BY mp.id, mp.name
HAVING COUNT(DISTINCT ti.tenant_id) > 1
LIMIT 5;
```

```
-- Resultado esperado:
-- Coca-Cola Original 1.5L | 7 | Minimarket Don Luis, Almacén El Rinconcito, ...
-- Pan Hallulla Ideal      | 5 | Minimarket Don Luis, Supermercado Familiar, ...
```

✅ Si ves productos maestros compartidos por múltiples tenants, la arquitectura está CORRECTA.

Verificación 7: Precios Diferentes por Tenant

```
-- Verificar que cada tenant tiene sus propios precios (aunque sea el mismo producto)

SELECT
  mp.name AS producto,
  t.business_name AS negocio,
  ti.cost_price AS costo,
  ti.sale_price AS venta,
  ti.stock
FROM master_products mp
JOIN tenant_inventory ti ON mp.id = ti.master_product_id
JOIN tenants t ON ti.tenant_id = t.id
WHERE mp.name LIKE '%Coca-Cola%'
      AND t.rut != '99.999.999-9'
ORDER BY mp.name, t.business_name;

-- Resultado esperado (precios diferentes):
-- Coca-Cola 1.5L | Minimarket Don Luis      | 900 | 1300 | 50
-- Coca-Cola 1.5L | Almacén El Rinconcito    | 850 | 1250 | 30
-- Coca-Cola 1.5L | Supermercado Familiar    | 920 | 1400 | 75
```

✓ Los precios deben ser DIFERENTES por tenant (esto es CORRECTO).

Verificación 8: Ventas por Tenant

```
-- Cada tenant debe tener ventas históricas
SELECT
  t.business_name,
  COUNT(s.id) AS total_ventas,
  SUM(s.total) AS total_facturado
FROM tenants t
LEFT JOIN sales s ON t.id = s.tenant_id
WHERE t.rut != '99.999.999-9'
GROUP BY t.id, t.business_name
ORDER BY total_facturado DESC;

-- Resultado esperado:
-- Minimarket Don Luis      | 187 | 2,345,600
-- Supermercado Familiar    | 234 | 3,456,800
-- Almacén El Rinconcito    | 98  | 1,234,500
-- ... (cada uno con 50-300 ventas)
```

Verificación 9: Items de Venta referencian Inventario del Tenant

```
-- Verificar que los items de venta apuntan al inventario del tenant correcto

SELECT
  s.sale_number,
  t.business_name AS tenant,
  si.quantity,
  mp.name AS producto,
  si.unit_price
FROM sales s
JOIN tenants t ON s.tenant_id = t.id
JOIN sale_items si ON s.id = si.sale_id
JOIN tenant_inventory ti ON si.tenant_inventory_id = ti.id
JOIN master_products mp ON ti.master_product_id = mp.id
WHERE s.tenant_id = (SELECT id FROM tenants WHERE rut = '76.123.456-7' LIMIT 1)
LIMIT 5;

-- Resultado esperado: ventas que referencian productos del inventario de ese tenant
```

✓ **NO debe haber ventas que referencien productos de otros tenants.**

8.2 Verificaciones en la Aplicación

Paso 1: Iniciar la aplicación

```
cd /home/ubuntu/CRTLpyme
npm run dev
```

Nota: Esto es localhost del servidor donde estás trabajando.

Paso 2: Login como PROVEEDOR (Admin SaaS)

- Email: `admin@crtlpyme.com`
- Contraseña: `Admin2025!`

Verificar:

- ✓ Dashboard muestra todos los tenants
- ✓ Puede ver ventas totales de la plataforma
- ✓ Puede filtrar por tenant específico
- ✓ Puede ver inventario de cualquier tenant
- ✓ Puede gestionar catálogo de productos maestros

Paso 3: Login como ADMIN de Tenant

- Email: `admin@minimarketdonluis.cl`
- Contraseña: `Demo2025!`

Verificar:

- ✓ Solo ve datos de su propio tenant
- ✗ NO puede ver datos de otros tenants

- ☒ Puede ver su inventario
 - ☒ Puede ver sus ventas
 - ☒ Puede buscar productos en el catálogo maestro
 - ☒ Puede agregar productos del maestro a su inventario
-

Paso 4: Verificar Landing Page (Planes de Suscripción)

- Ir a la landing page (URL pública de Cloud Run)
- Ir a la sección de planes

Verificar:

- ☒ Se muestran los tabs “Mensual” y “Anual”
- ☒ Al hacer clic en “Mensual” se ven 4 planes
- ☒ Al hacer clic en “Anual” se ven 4 planes
- ☒ Los precios son correctos
- ☒ Se muestra el badge “Ahorra 20%” en planes anuales

⚠ Si dice “No hay planes disponibles”, revisar:

1. ¿Los planes se crearon con `ANNUAL` o `YEARLY` ?
 2. ¿El frontend está filtrando por `ANNUAL` ?
-

9. RESUMEN Y RECOMENDACIONES

9.1 Problemas Críticos Identificados

#	Problema	Archivo Afectado	Severidad	Estado
1	Usa tabla <code>Product</code> (legacy) en lugar de <code>MasterProduct</code> / <code>TenantInventory</code>	<code>seed.ts</code>	● CRÍTICO	✗ Pendiente
2	Usa tabla <code>Product</code> (legacy)	<code>seed-multitenancy.ts</code>	● CRÍTICO	✗ Pendiente
3	Usa <code>YEARLY</code> en lugar de <code>ANNUAL</code>	<code>seed-subscription-plans.ts</code>	● MEDIO	✗ Pendiente
4	Archivo <code>productos_chile.json</code> no existe	<code>seed-complete.ts</code>	● CRÍTICO	✗ Pendiente
5	Seed por defecto ejecuta el incorrecto	<code>package.json</code>	● MEDIO	✗ Pendiente

9.2 Acciones Recomendadas

Prioridad ALTA (Hacer primero)

- ✓ Crear archivo `productos_chile.json`**
 - Ubicación: `/home/ubuntu/data/productos_chile.json`
 - Formato: JSON con estructura esperada
 - Mínimo: 100-200 productos chilenos básicos
- ✓ Corregir `seed-subscription-plans.ts`**
 - Cambiar `YEARLY` → `ANNUAL` (4 ocurrencias)
 - Ejecutar: `npm run seed:subscription-plans`
- ✓ Modificar `seed-complete.ts`**
 - Ajustar ruta del archivo de productos
 - Ajustar parseo del JSON (`data.productos`)
- ✓ Cambiar seed por defecto**
 - En `package.json` : `"seed": "tsx prisma/seed-complete.ts"`

5. Ejecutar seed completo

- Con flag `--clean` en desarrollo
- Sin flag en producción

Prioridad MEDIA (Hacer después)

1. Deprecar archivos de seed problemáticos

- Renombrar `seed.ts` → `seed-legacy.ts`
- Renombrar `seed-multitenancy.ts` → `seed-multitenancy-legacy.ts`
- Agregar comentarios de advertencia

2. Crear documentación de seed

- Explicar qué hace cada script
- Cuándo usar cada uno
- Cómo crear el archivo de productos

Prioridad BAJA (Mejoras futuras)

1. Crear script de verificación post-seed

- Ejecutar automáticamente después del seed
- Verificar separación multi-tenant
- Reportar problemas encontrados

2. Mejorar generación de datos

- Ventas más realistas (patrones por día/hora)
- Productos más variados por tipo de negocio
- Movimientos de inventario más complejos

9.3 Arquitectura Correcta (Resumen Visual)

ARQUITECTURA CORRECTA:

```

MasterProduct (Global, compartido)
  ↓ 1:N
TenantInventory (Por tenant, precios y stock propios)
  ↓ 1:N
SaleItem (Ventas aisladas por tenant)
  
```

ARQUITECTURA INCORRECTA (legacy):

```

Product (Por tenant, no compartido)
  ↓
No hay separación de catálogo vs inventario
  
```

9.4 Próximos Pasos

1. Revisar este documento completo
2. Decidir si usar `seed-complete.ts` (opción A) o crear uno nuevo (opción B)
3. Crear el archivo de productos maestros
4. Ejecutar las correcciones propuestas
5. Ejecutar el seed paso a paso

6. **Verificar con los queries SQL**
 7. **Probar en la aplicación**
 8. **Confirmar que todo funciona correctamente**
-

10. CONTACTO Y SOPORTE

Si tienes dudas o necesitas ayuda con la implementación:

-  Responde a este análisis con tus preguntas
-  Reporta problemas específicos que encuentres
-  Sugiere mejoras a este plan

Recuerda:

- No ejecutar comandos sin revisar primero
 - Hacer backup antes de limpiar datos
 - Probar en desarrollo antes de producción
-

Documento creado: 12 de Noviembre, 2025

Última actualización: 12 de Noviembre, 2025

Versión: 1.0

ANEXO: Comandos Útiles

```
# Conectar a Cloud SQL
gcloud sql connect ctrlpyme-db --user=postgres --project=ctrlpyme-477300

# Dentro de PostgreSQL:
\l                # Listar bases de datos
\c ctrlpyme_db    # Conectar a BD
\dt              # Listar tablas
\d+ tenants      # Describir tabla
\x              # Toggle expanded display
SELECT * FROM tenants;  # Ver tenants

# Salir de PostgreSQL
\q

# Ver logs de Cloud Run
gcloud run services logs read ctrlpyme --project=ctrlpyme-477300 --limit=50

# Ejecutar seed
cd /home/ubuntu/CRTLpyme
npm run seed:subscription-plans
npm run seed:complete -- --clean

# Verificar archivos
ls -lh /home/ubuntu/data/productos_chile.json
cat /home/ubuntu/data/productos_chile.json | jq '.productos | length'

# Editar archivos
nano prisma/seed-subscription-plans.ts
nano prisma/seed-complete.ts
```

FIN DEL DOCUMENTO